

students inputting their roll number. After verification, the users' attendance will be reconfirmed and stored in the database.

```
[from-pstn]
exten => 1234,1,Answer();
exten => 1234,n,Set(DID=${EXTEN});
exten => 1234,n,Playback(welcome);
exten => 1234,n,Playback(pls-enter-enroll);
exten => 1234,n,Read(enroll,beep,10);
exten => 1234,n,SayDigits(${enroll});
exten => 1234,n,Set(TIMEOUT(digit)=1);
exten => 1234,n,Set(TIMEOUT(response)=10);
exten => 1234,n,Background(pls-confirm);
exten => 1234,n,WaitExten()
```

The welcome message and the request for inputting the roll number is played. After that, the roll number is read up to 10 digits. Then the input digits are read out loud and a confirmation is requested.

```
exten => 1,1,NOOP(Caller confirmed entry);
exten => 1,n,Goto(auto-process,submenu,1);
exten => 2,1,NOOP(Caller wants to re-enter);
exten => 2,n,Goto(1234,3);
```

If the user confirms that the entry is correct, then the control proceeds to the auto-process sub-menu. Else, the control proceeds to re-enter the inputs.

```
[auto-process]
exten => submenu,1,Set(TIMEOUT(digit)=1);
exten => submenu,n,Set(TIMEOUT(response)=1);
exten => submenu,n,Background(Pls-select-frm-menu);
exten => submenu,n,WaitExten();
```

A request is made to the user to input the service needed. If the user wants to check the attendance so far, '1' can be pressed.

```
exten => 1,1,NOOP(Caller wants to check attendance);
exten => 1,n,MYSQL(Connect connid localhost Admin_DbUser
P@55_DbPass school_db);
exten => 1,n,MYSQL(Query resultid ${connid} SELECT
count(date) from attendance where enroll_no=${enroll} and
present="Y");
exten => 1,n,MYSQL(Fetch fetchid ${resultid} total);
exten => 1,n,MYSQL(Clear ${resultid});
exten => 1,n,MYSQL(Disconnect ${connid})
exten => 1,n,Playback(total-attendance-is);
exten => 1,n,SayDigits(${total});
exten => 1,n,goto(submenu,1);
```

When '1' is pressed, a NOOP statement is executed. A connection is made to the database with the correct user

name and password, and the total attendance is queried. The total attendance is then announced. Similarly, the program can execute any database operation including updation, deletion, etc.

As a good programmer, you will have already noticed that the program gets more and more unmanageable as the size grows. Also, the usage of *Goto* makes the program unreadable, as the size grows. There are multiple other ways to write IVR scripts like AGI-script (Asterisk Gateway Interface-script) or PHP. Let's explore the latter.

To write in PHP, download *phpagi2.2* from <http://phpagi.sourceforge.net/>. Unzip and keep the file inside */var/lib/asterisk/agi-bin*.

Now, in the dial plan, we can initiate the PHP script by including the following line:

```
[from-pstn]
exten => _.,1,AGI(welcome.php);
The following is the printout of welcome.php:
!usr/local/bin/php -q
<?php
set_time_limit(0);
require('phpagi.php');
$agi = new AGI();

$agi->answer();
$agi->verbose("Call Answered inside AGI script\n",3);
$agi->stream_file('welcome');
$agi->hangup();
?>
```

The PHP program also uses the AGI library. The script creates a new AGI object. It answers the incoming call, renders a verbose three-level message and plays the *welcome.gsm* file. After playing the message, the call is hung up. All the flexibility that PHP offers can be used in this mode. This includes database access, accessing third party APIs, etc.

The power of Asterisk as a platform for creating strong applications has been proven once again by doing so with simple programming.

Hardware

The hardware required is quite simple. You can use almost any Linux hardware with a PCI slot for a PRI card or FXO card. Asterisk needs to be installed on Linux hardware, and then the PCI card must be configured. Connect the PRI line from the service provider to the card, and you are ready to experience the power of Asterisk IVR! 

By: Devasia Kurian

The author is the founder and CEO of 'astTECS', an Asterisk software based company providing IP PBX, call centre diallers, etc. He can be contacted at d.kurian@asttecs.com.